Final Report

McCay Ruddick, Sophia Chang, Samiksha Karimbil

Introduction:

Our project focused on framing YouTube virality as a multi-class classification problem by predicting the specific "virality tier" of daily trending videos. Based on their engagement metrics, we categorized videos into four distinct classes: a_little_viral, pretty_viral, really_viral, and super_viral. We were primarily interested in this topic because we wanted to apply deep learning techniques to familiar, everyday data. This is a challenging problem because user engagement is highly subjective, and a video's success is heavily influenced by unpredictable factors such as YouTube's recommendation algorithm. Rather than trying to uncover the "secret formula" for creating a viral video, our objective was to evaluate how effectively different machine learning models could map numeric metadata and text features to these specific virality tiers. Throughout this project, we built and compared a wide variety of models, ranging from simple logistic regression to complex deep neural networks equipped with Natural Language Processing (NLP), to explore the trade-offs among model complexity, feature engineering, and computational cost.

Background

We came across two GitHub repositories that were quite similar to our project idea. The first repository utilized feature engineering to create multiple new columns such as title length, number of tags, and description length (ayush1997, n.d.). From its exploratory data analysis, it showed most of the view counts and like counts skewed right. They used a Random Forest algorithm for feature selection, with final features ending up being view count, comment count, dislike count, view count divided by video age in months, and subscriber count divided by video count. The overall conclusion was that the Random Forest model performance was fair, and at the time this research was done, there weren't other benchmarks to validate the results against (ayush1997, n.d.).

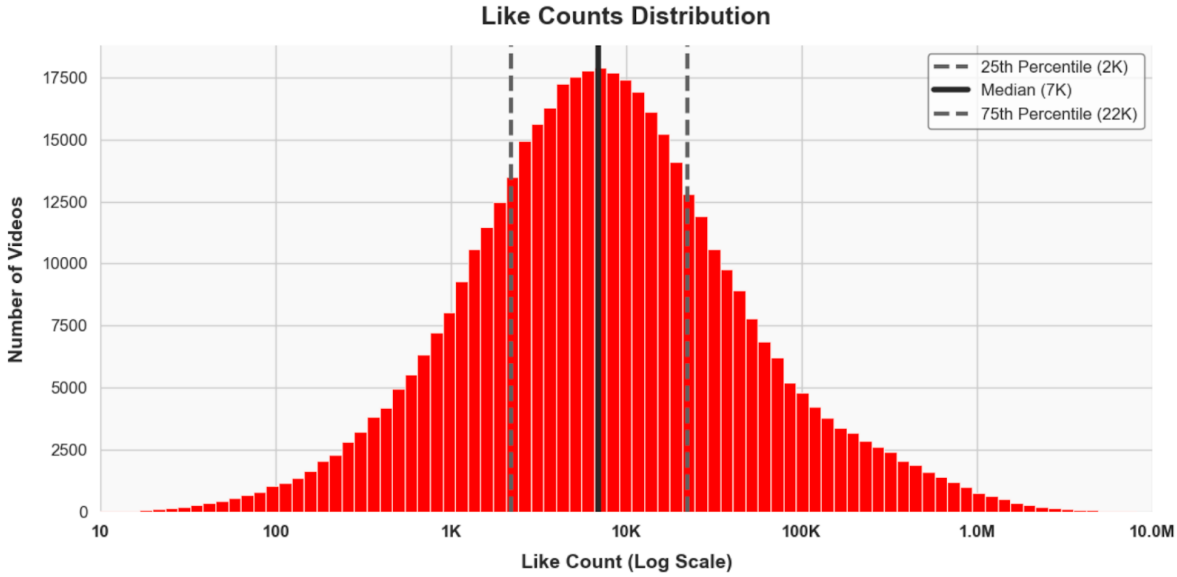
The second repository aimed to predict not only the number of video likes but also the likes-to-views ratio on YouTube (LionelKavit, n.d.). It used TF-IDF on video titles, descriptions, and tags, reducing dimensionality

with PCA. The authors used a feedforward neural network as their model with L1 loss. Using the Adam optimizer, they achieved a validation loss of 0.039. They utilized additional text features with TF-IDF and new categorical features, such as different languages (including Hindi and Arabic), which further improved overall accuracy. However, similar to the first repository, they faced challenges with highly skewed engagement data containing multiple outliers throughout the process (LionelKavit, n.d.).

Data and Methods

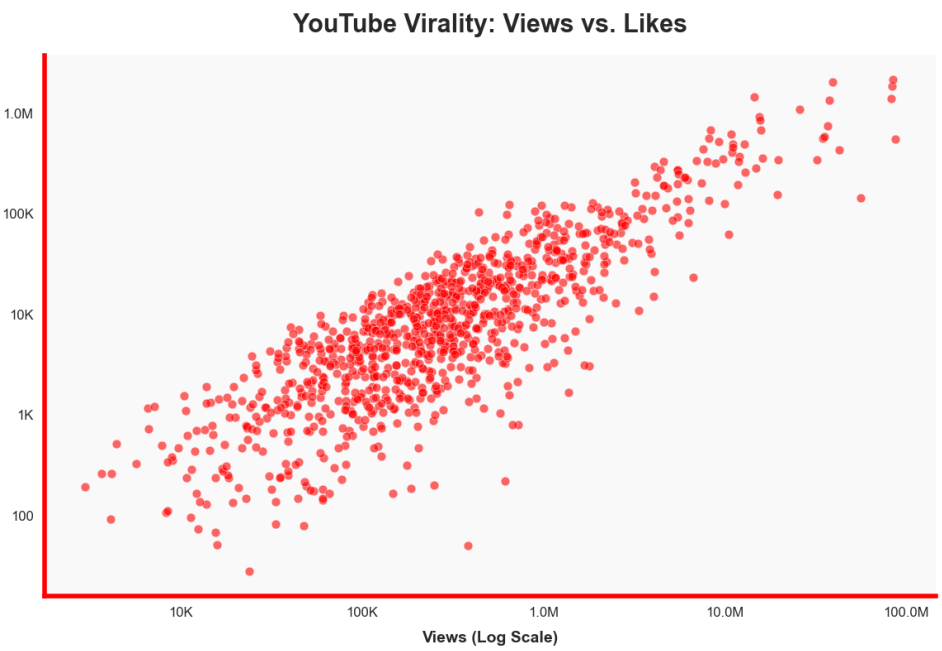
The dataset, sourced from Kaggle, contains weekly snapshots of YouTube's top 50 trending videos across 113 countries from September 2023 to February 2026 (Asaniczka, 2026). With over 4 million rows and 18 metadata columns per video, this provided a robust foundation for supervised learning. To prevent data leakage, we cleaned the data, removed duplicate video entries, and performed an 80/20 chronological train-test split.

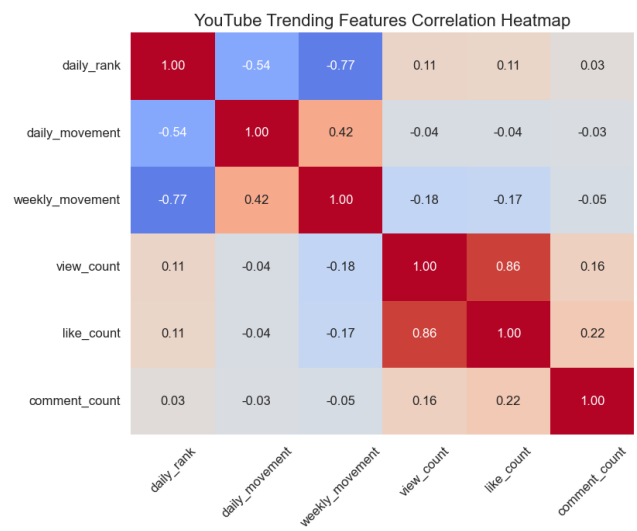
Before building any models, we conducted extensive Exploratory Data Analysis (EDA) to understand the dataset's underlying distributions and safely define our target variables. Plotting the distribution of like counts initially revealed a heavily right-skewed dataset; however, scaling the axis logarithmically transformed it into a clean normal distribution.



We used the 25th, median, and 75th percentiles from this normal distribution to strictly define our four classification tiers (a_little_viral, pretty_viral, really_viral, and super_viral).

We also generated a word cloud to visualize the most common words in trending titles. The prevalence of specific buzzwords confirmed our hypothesis that extracting textual features using Bag-of-Words (BoW) and TF-IDF would be valuable. Finally, we mapped views against likes. This scatter plot *(below)* showed a remarkably strong linear correlation on a log scale, establishing raw view count as the strongest and most logical heuristic for our baseline model.





Our implementation was structured as a multi-class classification problem. To systematically evaluate performance, we progressively built and tested a wide variety of models across four main categories:

- **Baselines:** Random Classifier, Majority Class Heuristic, and View Count Heuristic.
- **Logistic Regression:** Iterating from strictly numeric inputs to complex combinations including Scikit-Learn numeric processing, categorical data, BoW, and TF-IDF.
- **Multilayer Perceptrons (MLP):** Progressing from a purely numeric neural network to hybrid MLPs that ingested categorical features, NLP vectorization, and sentiment analysis.

- **Advanced Deep Architectures:** A Hybrid RNN combining Titles, Descriptions, and an MLP; an MLP utilizing Feature Attention; and a Pretrained LLM (DistilBERT) tested on both titles-only and all-tabular inputs.

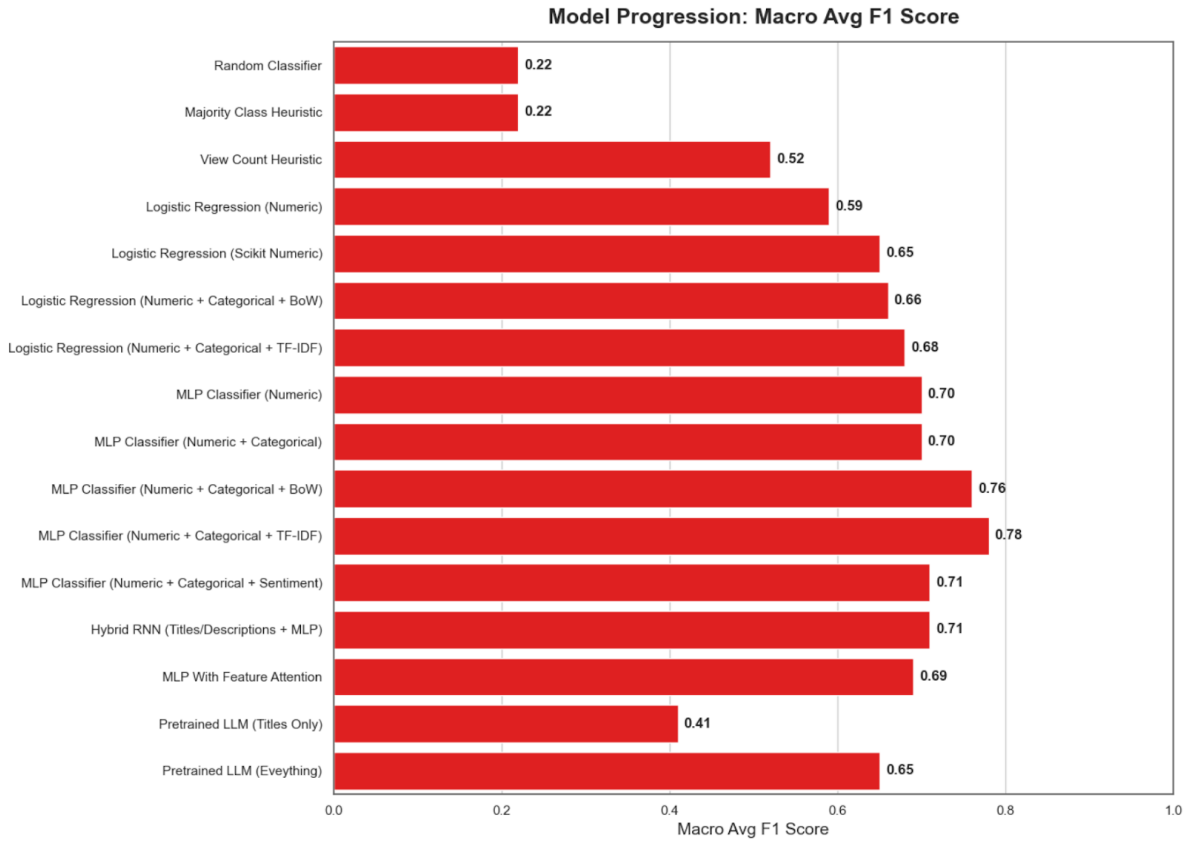
Evaluation

To evaluate our models, we used Scikit-Learn's classification_report to generate a single view of performance across all four virality tiers. The primary metrics were Precision, Recall, and the F1-Score, which are defined as follows:

- **Precision** = $\frac{TP}{TP+FP}$
- **Recall** = $\frac{TP}{TP+FN}$
- **F1-Score** = $2 \times \frac{Precision \times Recall}{Precision + Recall}$

We avoided using overall accuracy for this problem due to a massive class imbalance. Because 81.4% of our dataset belongs to the lowest virality tier, a model could achieve over 80% accuracy simply by guessing this baseline for every video without actually learning anything. To combat this, we selected the Macro-Averaged F1 Score as our primary metric. Because the Macro Average weights all four classes equally, it forced our models to identify rare traits of super viral videos (0.2% of the data) rather than taking the easiest statistical guess.

Below is the graph comparing the performance of all our models when evaluated under the Macro F1 metric.



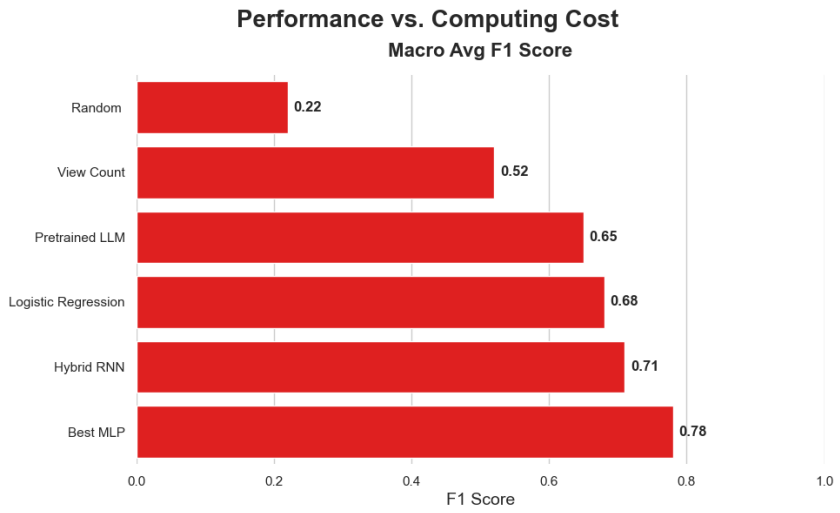
Results

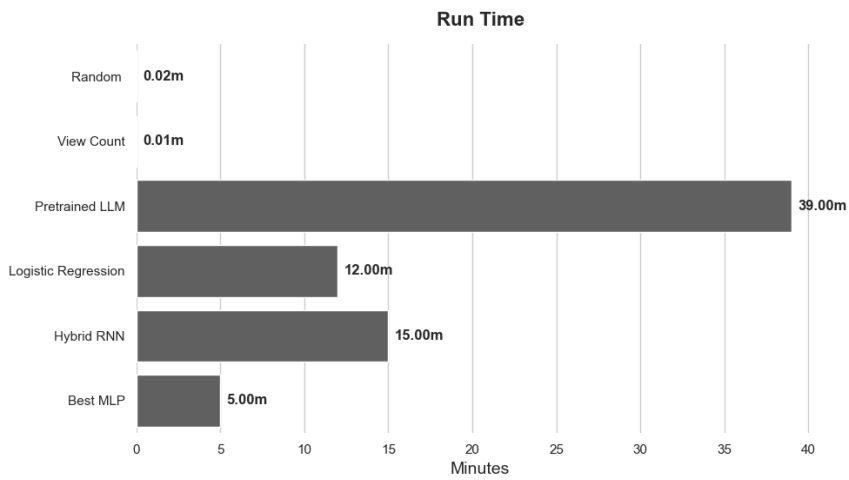
The first iteration of our project focused on establishing baselines and seeing how models could predict using strictly numeric features (daily rank, view count, comment count, etc.). Our baselines included a Random Classifier, Majority Class Heuristic, View Count Heuristic, and Logistic Regression. These showed promise, with the Logistic Regression model achieving a macro-averaged F1 score of 0.59 *(Note: this metric is used to score all models)*. However, these results showed significant room for improvement, especially in predicting high-virality tiers. Our first deep learning model was an MLP with numeric features and the Adam optimizer. This model did well, scoring 0.68, beating all the baseline models. While the numeric features were promising, they weren't capturing everything, and we felt we could do better by incorporating non-numeric features.

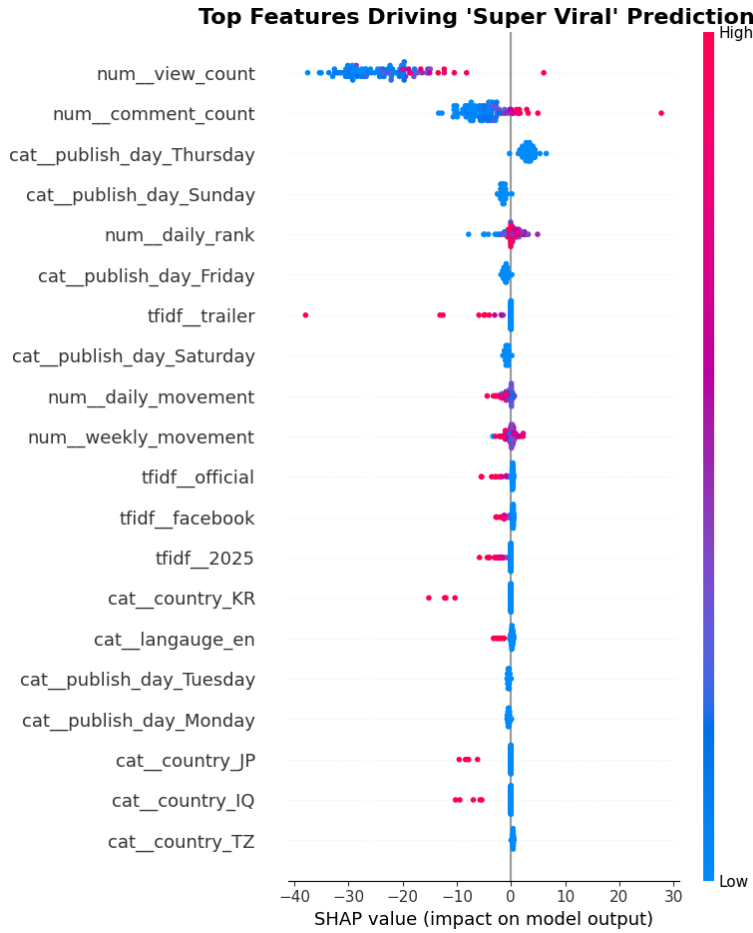
Our second Iteration was where we saw the most improvements in the entire project. In this iteration, we extracted categorical features (language, country, publish day) and vectorized video titles and descriptions using Bag-of-Words (BoW), TF-IDF, and Sentiment Analysis. These additions greatly improved our model. The MLP combined with TF-IDF scored a 0.78, becoming our best-performing model of the entire project. We were very happy with this score, but wanted to see if we could push it further with more complex models.

Our final Iteration involved adding Attention features to our MLP, creating an RNN model, and using DistilBERT, a pretrained LLM. However, these models failed to perform even remotely as well as our simpler MLP models. What's worse, the modes would at times take hours to train, while our simpler models would take at most a few minutes. They helped us conclude that the data, while complex, does not require a highly complex model to create predictions.

Below is a graph showing the performance of our best models at each iteration of our work, along with a feature-importance graph showing which features our best-performing model used to predict the highest level of virality.







Outlook

When we started this project, we expected that highly complex deep learning models, specifically recurrent neural networks and fancy pretrained models, would significantly outperform our simpler models. Because YouTube is a heavily content-driven platform, we anticipated that deep semantic understanding of titles and descriptions would be the key to predicting the highest virality tiers, the less common occurrences.

Our results only partially aligned with these expectations. While extracting text features via TF-IDF and feeding them into an MLP resulted in our absolute best-performing model, pushing the complexity any further yielded diminishing returns. Advanced architectures like the Hybrid RNN, Feature Attention networks, and DistilBERT failed to surpass our simpler TF-IDF MLP. Furthermore, as our SHAP analysis revealed, the model ultimately relied on raw numeric momentum (views, daily movement) rather than nuanced text features to successfully identify the rare *super_viral* videos.

The most valuable lesson we learned from this project was the massive trade-off between model complexity and practical computational cost. We saw firsthand that throwing a massive LLM at a tabular dataset can result in a multi-decade training time, only to perform worse than a 5-minute MLP. Navigating out-of-memory errors and hardware constraints gave us a nice, practical understanding of how real-world limitations must actively influence model selection.

If we were to continue this project, we would shift our focus away from building new architectures and instead fine-tune the hyperparameters of our winning TF-IDF MLP to maximize its accuracy. Ultimately, we would also want to extrapolate this model beyond the "Trending" tab, deploying it on standard, everyday videos to see if it can successfully predict a viral breakout before it even happens.

References

- Asaniczka. (2026). *Trending Youtube Video Statistics (113 Countries)* [Data set]. Kaggle. https://doi.org/10.34740/KAGGLE/DSV/14876253
- ayush1997. (n.d.). *AYUSH1997/YouTube-like-predictor: YouTube like count predictions using machine learning* [Source code]. GitHub. https://github.com/ayush1997/YouTube-Like-predictor
- LionelKavit. (n.d.). *Lionelkavit/YouTube-video-likes-predictor: This project uses a deep learning neural network to predict the likes-to-views ratio of a YouTube video* [Source code]. GitHub. https://github.com/LionelKavit/Youtube-Video-Likes-Predictor

Other

- Our project repository: https://github.com/Mcray3000/DeeplearningFinalUwU